

VLSI INTERNSHIP

WEEK 2 REPORT

Task 2: Introduction to Verilog HDL and RTL Design

RTL Design of Basic Combinational Circuits — Logic Gates, Half Adder & Full Adder

Name	Neel Vyas
College	G H Patel College of Engineering and Technology
Department	Electronics and Communication Engineering (ECE)
Intern ID	MT4900
Internship Duration	6 Weeks (14 May 2026 – 29 June 2026)
Submitted To	Maincrafts Technology
Date of Submission	May 2026

1. Introduction

This report documents the work completed for Task 2 of the VLSI Design Internship at Maincrafts Technology. Task 2 focuses on Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits, and represents my first practical step into actual VLSI implementation — moving from the conceptual understanding built in Task 1 to writing, simulating, and verifying real hardware description code.

In Task 1, I studied what VLSI is and where it is used, the complete VLSI design flow, digital logic fundamentals, and logic gates with their truth tables. Task 2 builds directly on that foundation by introducing the tool that industry engineers actually use to describe those circuits: Verilog Hardware Description Language (HDL). This task marks the transition from schematic-level thinking to code-based chip design.

The three circuits implemented are: a Logic Gates module (AND, OR, NOT, NAND, NOR, XOR), a Half Adder, and a Full Adder — all written in Verilog, simulated with Icarus Verilog, and waveform-verified in GTKWave.

2. Why This Task Matters — Industry Context

Before starting the actual implementation, I took time to understand why Verilog and RTL design are so central to the semiconductor industry. The key points I noted are:

- All digital ICs — from microcontrollers to high-performance CPUs — are described using Hardware Description Languages like Verilog or SystemVerilog. Schematic design is rarely used beyond basic education.
- RTL code is the starting point for the entire downstream VLSI flow: synthesis converts RTL to gates, physical design places and routes those gates, and timing analysis verifies the result. Everything depends on correct RTL.
- Companies expect VLSI interns and engineers to understand RTL concepts, write clean synthesisable Verilog, and verify functionality using testbenches. This task builds exactly that core skill.

Understanding this context helped me approach the coding exercises not as abstract assignments but as real engineering work — the same kind done at companies like Intel, Qualcomm, or TSMC design centres.

3. Concepts Studied

The following core concepts were studied before and during the implementation of Task 2:

Concept	Explanation
Verilog HDL	Hardware description language used to describe digital circuit structure and behaviour
RTL Design	Register Transfer Level modelling — the standard abstraction for synthesisable digital design
Module	The basic building block in Verilog; every circuit is described as a named module with ports
Continuous Assignment	The assign keyword creates a permanent combinational connection; output updates whenever input changes
Testbench	A non-synthesisable Verilog module that wraps the DUT, applies stimulus, and records outputs
Simulation	Functional validation of the design model using a simulator (Icarus Verilog) before hardware
VCD / GTKWave	Value Change Dump file records signal transitions; GTKWave displays them as waveform traces

3.1 What is Verilog? (My Understanding)

Verilog is a Hardware Description Language — not a programming language. The key difference is that Verilog describes concurrent hardware behaviour: multiple assign statements and always blocks execute simultaneously, just like real gates switching in parallel on a chip. In a software language like C, statements run one after another. In Verilog, six assign statements each compute their output at the same moment because the actual gates they model all switch at once.

The basic structure of any Verilog module is straightforward: declare the module name, list its input and output ports, describe the logic inside using assign or always blocks, and close with endmodule. Each module represents a distinct hardware block that can be instantiated (used) inside other modules — similar to how a chip can be plugged into a larger circuit board.

4. Tools Used and Simulation Workflow

Tool	Purpose	Type
Icarus Verilog (iverilog)	Compiles Verilog source files into a simulation executable	Free / Open Source
vvp	Executes the compiled simulation and produces the .vcd file	Bundled with Icarus
GTKWave	Waveform viewer — loads .vcd and displays signal traces	Free / Open Source

4.1 Simulation Steps (Icarus Verilog)

Each circuit was compiled, simulated, and viewed using the exact three-step command sequence below. The same workflow applies to all three designs — only the filenames change.

Half Adder

1. Compile design + testbench:

```
iverilog halfadder.v tb_half_adder.v
```

2. Run simulation (generates half_adder.vcd):

```
vvp a.out
```

3. Open waveform in GTKWave:

```
gtkwave half_adder.vcd
```

Full Adder

4. Compile design + testbench:

```
iverilog fulladder.v tb_full_adder.v
```

5. Run simulation (generates full_adder.vcd):

```
vvp a.out
```

6. Open waveform in GTKWave:

```
gtkwave full_adder.vcd
```

Logic Gates

7. Compile design + testbench:

```
iverilog logic_gates.v tb_logic_gates.v
```

8. Run simulation (generates logic_gates.vcd):

```
vvp a.out
```

9. Open waveform in GTKWave:

```
gtkwave logic_gates.vcd
```

What to verify after loading each waveform in GTKWave: outputs match the expected truth table, no unknown (X) states are present, and signal transitions occur at the correct simulation time steps.

5. Part A — Logic Gates Module

5.1 Design Logic

The Logic Gates module implements all six fundamental Boolean gates in a single Verilog module. Two shared inputs (A and B) feed all six gate expressions simultaneously. Each output is driven by one continuous assignment — the most direct mapping of a Boolean expression to hardware that Verilog offers.

Output Signal	Operator	Boolean Expression	Gate Behaviour
AND_Y	&	A & B	High only when A=1 AND B=1
OR_Y		A B	High when A=1 OR B=1 (or both)
NOT_Y	~	~A	Inverts A; output is complement of A
NAND_Y	~(A & B)	~(A & B)	Low only when A=1 AND B=1 (universal gate)
NOR_Y	~(A B)	~(A B)	High only when both A=0 AND B=0 (universal gate)
XOR_Y	^	A ^ B	High only when A ≠ B

5.2 Truth Table

A	B	AND_Y	OR_Y	NOT_Y	NAND_Y	NOR_Y	XOR_Y
0	0	0	0	1	1	1	0
0	1	0	1	1	1	0	1
1	0	0	1	0	1	0	1
1	1	1	1	0	0	0	0

Table 1 — Logic Gates Truth Table (all four input combinations)

5.3 Verilog Module Code (logic_gates.v)

The module uses six assign statements — one per gate. Because these are continuous assignments, all six outputs update simultaneously whenever A or B changes, exactly as real gates would behave on silicon:

```
module logic_gates (  
    input A,  
    input B,  
    output AND_Y,  
    output OR_Y,  
    output NOT_Y,  
    output NAND_Y,  
    output NOR_Y,  
    output XOR_Y,  
);  
  
    assign AND_Y = A & B;  
    assign OR_Y = A | B;  
    assign NOT_Y = ~A;  
    assign NAND_Y = ~(A & B);  
    assign NOR_Y = ~(A | B);  
    assign XOR_Y = A ^ B;  
endmodule
```

```

        output XOR Y
    );
    assign AND_Y = A & B;           // AND gate
    assign OR_Y  = A | B;           // OR gate
    assign NOT_Y = ~A;              // NOT gate (inverts A only)
    assign NAND_Y = ~(A & B);       // NAND = complement of AND
    assign NOR_Y  = ~(A | B);       // NOR  = complement of OR
    assign XOR_Y  = A ^ B;          // XOR gate
endmodule

```

5.4 Testbench Code (tb_logic_gates.v)

The testbench declares A and B as reg (driven procedurally) and all outputs as wire (driven by the module). It cycles through the four input combinations with 10 ns between each, dumps the waveform to logic_gates.vcd, and calls \$finish to end simulation:

```

module tb_logic_gates;
    reg  A, B;
    wire AND_Y, OR_Y, NOT_Y, NAND_Y, NOR_Y, XOR_Y;

    // Instantiate the DUT (Device Under Test)
    logic_gates uut (
        .A(A),          .B(B),
        .AND_Y(AND_Y), .OR_Y(OR_Y), .NOT_Y(NOT_Y),
        .NAND_Y(NAND_Y), .NOR_Y(NOR_Y), .XOR_Y(XOR_Y)
    );

    initial begin
        $dumpfile("logic_gates.vcd"); // waveform output file
        $dumpvars(0, tb_logic_gates); // dump all signals

        A = 0; B = 0; #10; // Test 1: A=0 B=0
        A = 0; B = 1; #10; // Test 2: A=0 B=1
        A = 1; B = 0; #10; // Test 3: A=1 B=0
        A = 1; B = 1; #10; // Test 4: A=1 B=1
        $finish;
    end
endmodule

```

5.5 GTKWave Waveform — Simulation Screenshot

The screenshot below was captured from GTKWave after loading logic_gates.vcd. The eight signal traces span 40 ns across four 10-ns windows, one per test case:

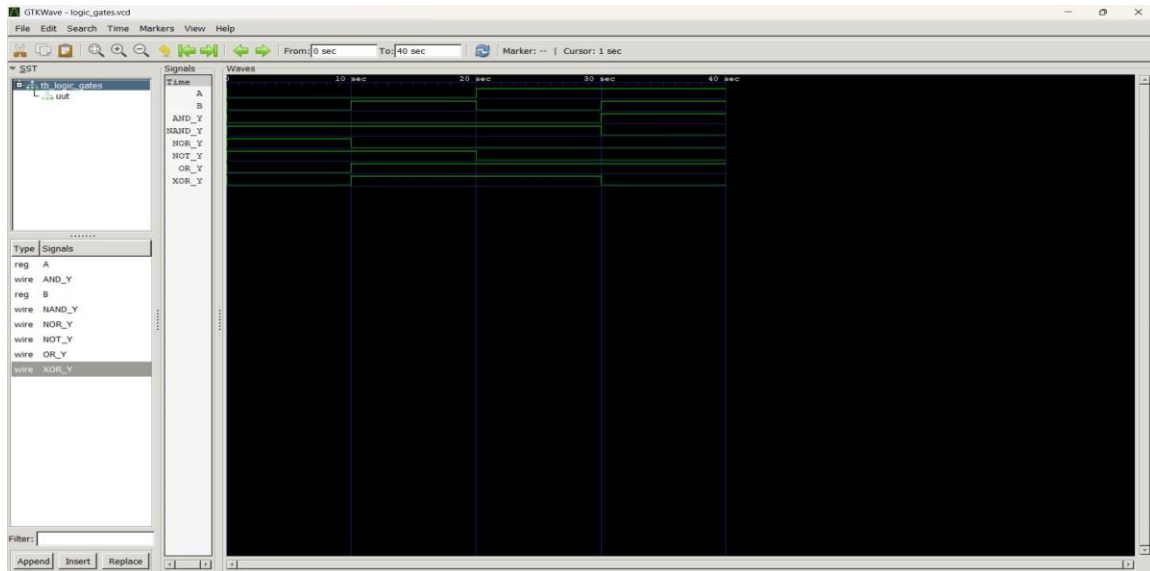


Figure 1 — GTKWave: Logic Gates simulation (logic_gates.vcd) — 40 ns, 4 test cases

5.6 Waveform Analysis and Observations

Reading Figure 1 left to right across the four 10-ns windows:

- 0–10 ns (A=0, B=0): All 'active-high' outputs (AND, OR, XOR) are 0; all 'active-low' complement outputs (NAND, NOR) are 1; NOT_Y is 1 because A=0. Matches row 1 of truth table exactly.
- 10–20 ns (A=0, B=1): OR_Y and XOR_Y rise to 1. NAND stays 1. NOR falls to 0 because OR is now 1. NOT_Y stays 1 (still inverting A=0). AND stays 0.
- 20–30 ns (A=1, B=0): OR_Y and XOR_Y stay 1. NOT_Y falls to 0 — this is the first moment A=1, so its inversion becomes 0. NAND and AND unchanged. NOR stays 0.
- 30–40 ns (A=1, B=1): AND_Y rises to 1 (first time both inputs are 1). NAND_Y and XOR_Y both fall to 0. OR_Y stays 1. NOT_Y stays 0. NOR stays 0.

Key observation: NAND_Y is the exact inversion of AND_Y at every time step, and NOR_Y is the exact inversion of OR_Y — this visually confirms De Morgan's theorem in hardware. XOR behaves as a 'difference detector', going High only when inputs differ.

6. Part B — Half Adder (RTL)

6.1 Design Logic

A Half Adder adds two single-bit binary numbers (A and B) and produces two outputs: Sum (the result bit) and Carry (the overflow bit). It is called 'half' because it has no provision for a carry-in from a previous bit position — the Full Adder in Part C adds that capability.

The design equations are derived directly from the binary addition truth table:

- $\text{Sum} = A \text{ XOR } B$ — The result is 1 only when the inputs differ, which is exactly the XOR function. When both are 1, XOR gives 0 (the sum wraps around to 0 in binary).
- $\text{Carry} = A \text{ AND } B$ — A carry is produced only when both inputs are 1, since $1+1=10$ in binary and the '1' in the tens position is the carry. AND is the exact gate for this.

These two equations require exactly two gates — one XOR and one AND — sharing the same pair of inputs. The elegance of the Half Adder is that the two simplest arithmetic gates (XOR and AND) completely describe binary single-bit addition.

6.2 Truth Table

A (Input)	B (Input)	Sum (A XOR B)	Carry (A AND B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 2 — Half Adder Truth Table

6.3 Verilog Module Code (halfadder.v)

The module maps directly to the two equations. Each assign statement corresponds to one physical gate:

```
module half_adder (
    input A,
    input B,
    output Sum,
    output Carry
);
    assign Sum    = A ^ B;    // XOR gate - computes the sum bit
    assign Carry  = A & B;    // AND gate - computes the carry bit
endmodule
```

6.4 Testbench Code (tb_half_adder.v)

The testbench instantiates half_adder as uut and steps through all four input combinations. Note that A and B are declared as reg (driven inside the initial block) while Sum and Carry are wire (driven by the DUT's assign statements):

```
module tb_half_adder;
    reg A, B;
    wire Sum, Carry;
```

```
// Instantiate Half Adder as 'uut' (unit under test)
half adder uut (
    .A(A),
    .B(B),
    .Sum(Sum),
    .Carry(Carry)
);

initial begin
    $dumpfile("half_adder.vcd");
    $dumpvars(0, tb half adder);

    A = 0; B = 0; #10;    // 0 + 0 = 0, carry 0
    A = 0; B = 1; #10;    // 0 + 1 = 1, carry 0
    A = 1; B = 0; #10;    // 1 + 0 = 1, carry 0
    A = 1; B = 1; #10;    // 1 + 1 = 10 -> Sum=0, Carry=1
    $finish;
end
endmodule
```

6.5 GTKWave Waveform — Simulation Screenshot

The waveform below (loaded from half_adder.vcd) shows A, B, Carry, and Sum across 40 ns. Four 10-ns windows map to the four test cases:

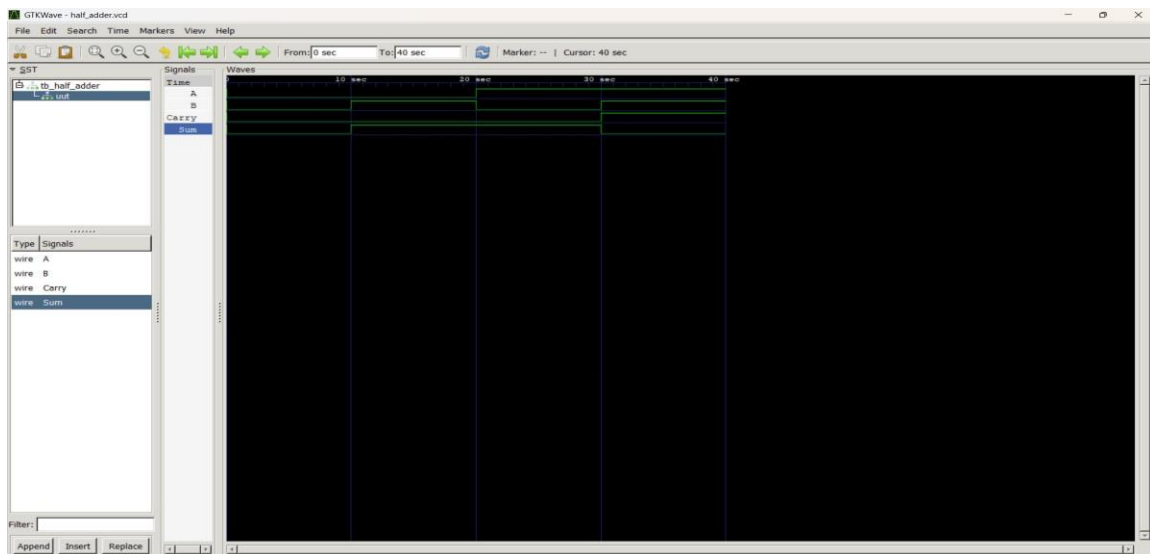


Figure 2 — GTKWave: Half Adder simulation (half_adder.vcd) — 40 ns, 4 test cases

6.6 Waveform Analysis and Observations

Figure 2 shows four signal traces. The simulation results match the truth table exactly:

- 0–10 ns (A=0, B=0): Sum=0, Carry=0. Nothing to add; both outputs stay Low.
- 10–20 ns (A=0, B=1): Sum rises to 1, Carry stays 0. Binary $0+1=01$; sum bit is 1, no overflow.

- 20–30 ns (A=1, B=0): Sum stays 1, Carry stays 0. Binary 1+0=01; same arithmetic result but different input source.
- 30–40 ns (A=1, B=1): Sum falls to 0, Carry rises to 1. This is the most important case — binary 1+1=10 (decimal 2). The XOR gate correctly produces 0 for equal inputs, while the AND gate produces 1 for the overflow. The two bits together (Carry=1, Sum=0) encode the binary value '10', which is 2.

Personal observation: Without the Carry output, a half adder would silently give the wrong answer for A=B=1. This is why two outputs are essential, and why the AND gate is as critical as the XOR gate in the design.

7. Part C — Full Adder (RTL)

7.1 Design Logic

The Full Adder solves the limitation of the Half Adder by adding a third input: Cin (Carry-in). This allows multiple Full Adders to be chained together in a ripple-carry configuration to add binary numbers of any bit width — the same principle used in ALUs inside every CPU.

A Full Adder accepts A, B, and Cin and produces Sum and Carry (Cout). The Boolean equations are:

- $\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$ — The double-XOR correctly produces 1 whenever an odd number of the three inputs are 1, and 0 when an even number are 1.
- $\text{Carry} = (A \text{ AND } B) \text{ OR } (B \text{ AND } \text{Cin}) \text{ OR } (A \text{ AND } \text{Cin})$ — This is the majority function: a carry is generated whenever any two or more of the three inputs are 1. Each AND term detects one pair; the OR combines them.

Structurally, a Full Adder can be built from two Half Adders plus an OR gate. The first Half Adder computes (A XOR B) and (A AND B). The second adds Cin to the partial sum. The OR gate merges both partial carries. In RTL, this is written more efficiently as the two assign statements below.

7.2 Truth Table

A	B	Cin	Sum	Carry (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1

1	1	0	0	1
1	1	1	1	1

Table 3 — Full Adder Truth Table (all 8 input combinations)

7.3 Verilog Module Code (fulladder.v)

Two continuous assignments implement the complete Full Adder. The double-XOR for Sum and the three-product OR for Carry are the standard RTL formulation used in industry:

```
module full_adder (
    input  A,
    input  B,
    input  Cin,
    output Sum,
    output Carry
);
    // Double-XOR for sum: 1 when odd number of inputs are 1
    assign Sum    = A ^ B ^ Cin;

    // Majority function for carry: 1 when any two (or all) inputs are 1
    assign Carry = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

7.4 Testbench Code (tb_full_adder.v)

The testbench covers all eight input combinations (000 to 111) with 10 ns between each, giving 80 ns total simulation time:

```
module tb_full_adder;
    reg  A, B, Cin;
    wire Sum, Carry;

    // Instantiate Full Adder
    full_adder uut (
        .A(A), .B(B), .Cin(Cin),
        .Sum(Sum), .Carry(Carry)
    );

    initial begin
        $dumpfile("full_adder.vcd");
        $dumpvars(0, tb_full_adder);

        A=0; B=0; Cin=0; #10; // 0+0+0 = 00 (Sum=0, Carry=0)
        A=0; B=0; Cin=1; #10; // 0+0+1 = 01 (Sum=1, Carry=0)
        A=0; B=1; Cin=0; #10; // 0+1+0 = 01 (Sum=1, Carry=0)
        A=0; B=1; Cin=1; #10; // 0+1+1 = 10 (Sum=0, Carry=1)
        A=1; B=0; Cin=0; #10; // 1+0+0 = 01 (Sum=1, Carry=0)
        A=1; B=0; Cin=1; #10; // 1+0+1 = 10 (Sum=0, Carry=1)
    end
endmodule
```

```

    A=1; B=1; Cin=0; #10; // 1+1+0 = 10 (Sum=0, Carry=1)
    A=1; B=1; Cin=1; #10; // 1+1+1 = 11 (Sum=1, Carry=1)
    $finish;
end
endmodule

```

7.5 GTKWave Waveform — Simulation Screenshot

The waveform shows five signals (A, B, Cin, Carry, Sum) across 80 ns. Each 10-ns segment corresponds to one row of the truth table:

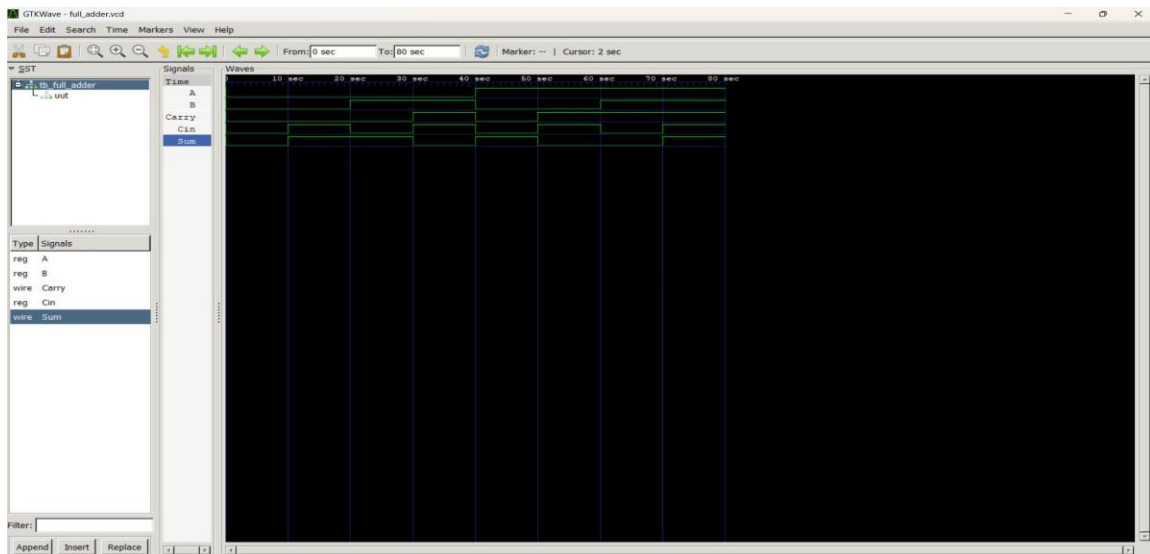


Figure 3 — GTKWave: Full Adder simulation (full_adder.vcd) — 80 ns, 8 test cases

7.6 Waveform Analysis and Observations

Figure 3 shows all eight test cases verified against the truth table:

- 0–10 ns (0,0,0): Sum=0, Carry=0. Three zeros sum to zero with no carry.
- 10–20 ns (0,0,1): Sum=1, Carry=0. Only Cin is 1; result is 1, no carry.
- 20–30 ns (0,1,0): Sum=1, Carry=0. Only B is 1; same result as previous.
- 30–40 ns (0,1,1): Sum=0, Carry=1. Two inputs are 1 — majority function triggers carry. 0+1+1=10 binary.
- 40–50 ns (1,0,0): Sum=1, Carry=0. Only A is 1; result is 1.
- 50–60 ns (1,0,1): Sum=0, Carry=1. A and Cin both 1 — majority function triggers carry. 1+0+1=10 binary.
- 60–70 ns (1,1,0): Sum=0, Carry=1. A and B both 1 — (A AND B) term alone drives carry. 1+1+0=10 binary.
- 70–80 ns (1,1,1): Sum=1, Carry=1. All three inputs are 1 — 1+1+1=11 binary (decimal 3). Both Sum AND Carry are simultaneously High. This is the maximum possible output of a single-bit Full Adder and confirms the design handles the 'all ones' edge case correctly.

The most instructive observation from this simulation is the Carry pattern: it asserts any time two or more inputs are High, regardless of which two. This is the majority function in action — clearly visible across the eight time windows.

8. Key Learning Outcomes

After completing Task 2, I am now able to:

- Write synthesisable Verilog code using modules, ports, and continuous assignments for combinational logic.
- Design combinational circuits at RTL level — translating Boolean equations directly into Verilog assign statements.
- Create complete testbenches: declare reg/wire correctly, instantiate the DUT, apply stimulus with `#delays`, and dump waveforms.
- Compile and simulate Verilog using the `iverilog → vvp → gtkwave` toolchain from the command line.
- Read and interpret GTKWave waveforms: match signal transitions to test cases and verify against truth tables.
- Understand the difference between a Half Adder (two inputs) and Full Adder (three inputs, cascade-capable), and why both are essential building blocks in arithmetic hardware.
- Prepare for synthesis-level tasks: all three modules written here are synthesisable and could be mapped to a gate-level netlist in the next stage of the VLSI flow.

9. Deliverables Completed

The following mandatory deliverables specified in Task 2 have been completed and documented in this report:

Deliverable	Status	Location in Report
logic_gates.v — Verilog source file	Completed	Section 5.3
halfadder.v — Verilog source file	Completed	Section 6.3
fulladder.v — Verilog source file	Completed	Section 7.3
tb_logic_gates.v — Testbench file	Completed	Section 5.4
tb_half_adder.v — Testbench file	Completed	Section 6.4
tb_full_adder.v — Testbench file	Completed	Section 7.4
Simulation screenshot — Logic Gates	Completed	Section 5.5 (Figure 1)
Simulation screenshot — Half Adder	Completed	Section 6.5 (Figure 2)
Simulation screenshot — Full Adder	Completed	Section 7.5 (Figure 3)

Design logic explanation	Completed	Sections 5.1, 6.1, 7.1
Simulation results analysis	Completed	Sections 5.6, 6.6, 7.6

Optional deliverables noted in Task 2: inline comments explaining each line of code have been added to all module and testbench listings throughout this report.

10. Conclusion

Task 2 of the VLSI Internship has been a genuinely practical step — the transition from reading about digital logic to actually coding, compiling, and simulating it. Writing the three Verilog modules and their testbenches built a strong intuition for how hardware description languages differ from software programming: every assign statement is a wire, not a command, and all wires carry their values simultaneously.

The Logic Gates module made this concurrency tangible — six assign statements producing six simultaneous outputs from the same two inputs, just as six real gates would. The Half Adder showed how exactly two gates are sufficient to describe binary addition, and why losing the Carry output would mean losing information. The Full Adder extended that understanding to three-input addition and introduced the concept of cascadeable arithmetic circuits — the foundation of all multi-bit adders in modern ALUs.

GTKWave was an invaluable learning tool. Reading waveforms forced me to think about time-ordered signal transitions rather than just static truth tables, which is much closer to how engineers reason about real circuit behaviour under timing constraints.

The skills developed in Task 2 — RTL coding, testbench writing, simulation, and waveform analysis — are directly applicable to the more advanced designs in the upcoming tasks, and form the core toolkit required for all synthesis and physical design work in the VLSI flow.

11. References

- Icarus Verilog — Open-source Verilog simulator: <http://iverilog.icarus.com>
- GTKWave — Waveform viewer: <http://gtkwave.sourceforge.net>
- TutorialsPoint — Verilog Tutorial: https://www.tutorialspoint.com/vlsi_design/vlsi_verilog.htm
- GeeksForGeeks — Digital Logic Design: <https://www.geeksforgeeks.org/digital-electronics-digital-logic/>